

0.91.6. Le snake — la queue grandit à chaque pièce

Le 0.91.5, mais la queue est un tableau de cases O. Au début, le A est tout seul ; chaque pièce ramassée ajoute un O. Le serpent grandit.

1. Ce qu'il faut comprendre

C'est le 0.91.5, avec une chose en plus : la queue grandit. Au début, le A est **tout seul**. À chaque P ramassé, il gagne **une case O**. Au bout de dix pièces, le A traîne dix O derrière lui.

Une queue de plusieurs cases, c'est plusieurs places : un tableau. `uint8_t qx[50];` réserve **50** colonnes, de `qx[0]` à `qx[49]` ; `qy[50]`, les 50 lignes. La case `i` de la queue est en `(qx[i], qy[i])`. `qx[0]` est la case collée au A ; la dernière est le **bout** de la queue.

longueur compte les cases utilisées : **0** au départ, le A est tout seul ; `nouvellePartie()` la remet à 0 et ne pose aucun O. Les cases utilisées vont de `0` à `longueur - 1` : avec `longueur = 3`, ce sont `qx[0]`, `qx[1]` et `qx[2]`. Le bout est donc toujours `qx[longueur - 1]`.

Un piège : longueur - 1 quand longueur vaut 0. On attendrait -1. Mais un `uint8_t` ne connaît que 0 à 255 : sous 0, il repart de l'autre côté, et 0 - 1 donne **255**. `qx[255]` n'existe pas (le tableau s'arrête à `qx[49]`), et la boucle ferait 255 tours. D'où la condition `longueur > 0 && (x != ax || y != ay)` : `&&` veut dire « et » (le 0.75) ; **les deux** doivent être vraies. Sans queue, on ne la déplace pas.

Les parenthèses autour de `x != ax || y != ay` : elles se calculent d'abord, comme en maths. « Il y a une queue » ET « le A a bougé (en x ou en y) ».

Ce qui est nouveau ici : faire avancer toute la queue. Chaque case prend la place de **celle de devant** : `qx[2]` prend la place de `qx[1]`, puis `qx[1]` celle de `qx[0]`, puis `qx[0]` celle que le A vient de quitter (`ax`, `ay`). Comme les wagons d'un train.

Pourquoi en partant du bout ? Si l'on commençait par `qx[1] = qx[0];`, l'ancienne place de `qx[1]` serait perdue avant que `qx[2]` ne la prenne. En partant du bout, chaque case est lue **avant** d'être écrasée.

La boucle qui recule : `for (uint8_t i = longueur - 1; i > 0; i--)`. `i--` retire 1 à `i` à chaque tour (le contraire de `i++`). Avec `longueur = 3` : `i` vaut 2, puis 1, et s'arrête avant 0 (`i > 0` est faux). Tour `i = 2` : `qx[2] = qx[1]`. Tour `i = 1` : `qx[1] = qx[0]`. Avec `longueur = 1`, `i` part de 0 : `0 > 0` est faux, la boucle ne tourne pas du tout.

Le bout est effacé avant, toute la queue est redessinée après : `effacer(qx[longueur - 1], qy[longueur - 1], 1)` gomme l'ancien bout, puis une boucle `for` pose un O sur chaque case, de `0` à `longueur - 1`. Et le A par-dessus, comme au 0.91.5.

Grandir, quand on ramasse le P : la nouvelle case naît **sur le bout de la queue** : `qx[longueur] = qx[longueur - 1];`, puis `longueur = longueur + 1;`. Au pas suivant, toute la queue avance d'un cran, sauf la nouvelle case, qui prend la place de l'ancien bout : **elle reste derrière**, et la queue a une case de plus.

La toute première case, elle, n’a pas de bout sur lequel naître (et `qx[longueur - 1]` serait encore `qx[255]`). Elle naît donc **sous le A** : `qx[0] = x;` et `qy[0] = y;`. Au pas suivant, le A s’en va, et le O apparaît sur la case qu’il a quittée. `else` (le 0.83.1) : « sinon », quand la queue a déjà au moins une case.

Pourquoi redessiner toute la queue ? Juste après la pièce, le bout et la nouvelle case sont sur la même place. Au pas suivant, on efface le bout... donc aussi la nouvelle case, qui doit y rester. Redessiner toutes les cases la fait réapparaître.

`if (longueur < 50)` : le tableau n’a que 50 cases. Au-delà, `qx[50]` écrirait en dehors, sur d’autres variables. Après 49 pièces, la queue ne grandit plus.

Pour plus tard : le A traverse sa propre queue sans rien dire, et le P peut tomber sur la queue. Dans un vrai serpent, toucher sa queue fait perdre.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <couleurTexte>` → [0.87.7. La fonction couleurTexte\(\) — la couleur des lettres](#)
- `#include <texteGrand>` → [0.86.3. La fonction texteGrand\(\) — un texte agrandi](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <deplace_croix>` → [0.66.3. La fonction deplace_croix\(\) — suivre la croix, case par case](#)
- `#include <hasard>` → [0.81.2. La fonction hasard\(\) — un nombre au hasard](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l’œil le lit. Il fait **2646 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l’a demandé — et ce qu’il a laissé, parce que rien ne l’appelle :

Gravé dans la cartouche :

démarrage	117 o	← la console, à l’allumage — la mise en route de la console
texte_grand_0()	195 o	← main() — ta fonction
chaque_minuteur()	124 o	← main() — une fonction fournie par le langage
deplace_croix()	217 o	← main() — une fonction fournie par le langage
viderEcran()	33 o	← nouvellePartie(), main() — ta fonction
nouvellePartie()	163 o	← main() — ta fonction
main()	619 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l’image suivante
AttendreVBlank	16 o	← démarrage — attend que l’écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer() — trouve l’adresse d’une case de l’écran
EffacerCases	12 o	← effacer() — efface des cases
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
Reste	10 o	← main() — le reste d’une division
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l’écran

Hasard	25 o	← hasard()	– tire un nombre au hasard
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	848 o	← CopierTuiles	– la police entière, et 9 tuiles dessinées
"SCORE"	5 o	← texte()	– un texte gravé
"TEMPS"	5 o	← texte()	– un texte gravé
"APPUIE SUR START"	16 o	← texte()	– un texte gravé
"FIN"	3 o	← texte()	– un texte gravé
"SCORE"	5 o	← texte()	– un texte gravé
"START : REJOUER"	15 o	← texte()	– un texte gravé
GRAND_TABLE_0_0_0	12 o	← poser()	– une table gravée
GRAND_TABLE_0_1_1	12 o	← poser()	– une table gravée
GRAND_TABLE_0_2_2	12 o	← poser()	– une table gravée
GRAND_TABLE_0_3_3	12 o	← poser()	– une table gravée

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 15	278 o sur place, dans texte_grand_0(), deplace_croix(), nouvellePartie(), main() + AdresseCase (10 o), GRAND_TABLE_0_0_0 (12 o), GRAND_TABLE_0_1_1 (12 o), GRAND_TABLE_0_2_2 (12 o), GRAND_TABLE_0_3_3 (12 o)
texte()	× 6	66 o sur place, dans nouvellePartie(), main() + EcrireTexte (10 o)
effacer()	× 3	64 o sur place, dans deplace_croix(), viderEcran(), main() + AdresseCase (10 o), EffacerCases (12 o)
bouton()	× 6	30 o sur place, dans deplace_croix(), main() + LireManette (37 o)
nombre()	× 3	30 o sur place, dans main() + EcrireNombre (33 o)
couleurFond()	× 1	12 o sur place, dans main()
hasard()	× 2	6 o sur place, dans main() + Hasard (25 o)
images()	× 2	4 o sur place, dans chaque_minuteur(), deplace_croix()
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)

Tes fonctions appelées :

deplace_croix()	× 1	19 o sur place, dans main()
chaque_minuteur()	× 1	10 o sur place, dans main()
viderEcran()	× 2	6 o sur place, dans nouvellePartie(), main()
nouvellePartie()	× 2	6 o sur place, dans main()
texte_grand_0()	× 1	3 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <reste>    // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair
#include <effacer>    // efface des cases, ou tout le fond
#include <poser>    // pose une tuile sur une case du fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <texte>    // écrit un texte à l'écran
#include <couleurTexte>    // choisit la couleur des lettres
#include <texteGrand>    // écrit un texte agrandi, de 1 à 20 fois
#include <bouton>    // lit un bouton de la manette
#include <deplace_croix>    // une tuile qui suit la croix, case par case
#include <hasard>    // tire un nombre au hasard
#include <nombre>    // écrit un nombre en chiffres
#include <chaque>    // répond 1 toutes les n millisecondes
```

2. UNE VARIABLE — ECRAN

```
uint8_t ecran = 0;    // 0 = le TITRE, 1 = le JEU, 2 = la FIN
```

3. UNE VARIABLE — X

```
uint8_t x = 5;    // le A : sa colonne...
```

4. UNE VARIABLE — Y

```
uint8_t y = 8;    // ...et sa ligne (le 0.81)
```

5. UNE VARIABLE — PX

```
uint8_t px = 15;          // la pièce, le P : sa colonne...
```

6. UNE VARIABLE — PY

```
uint8_t py = 8;          // ...et sa ligne
```

7. UNE VARIABLE — SCORE

```
uint8_t score = 0;       // les pièces ramassées
```

8. UNE VARIABLE — TEMPS

```
uint8_t temps = 30;      // les secondes qui restent
```

9. UNE VARIABLE — AX

```
uint8_t ax = 5;          // la place du A AVANT son pas... (le 0.91.3)
```

10. UNE VARIABLE — AY

```
uint8_t ay = 8;          // ...pour l'y remettre s'il entre dans un mur
```

11. UN TABLEAU — QX

```
uint8_t qx[50];          // NOUVEAU : la queue, un TABLEAU de 50 cases 0 : les colonnes...
```

12. UN TABLEAU — QY

```
uint8_t qy[50];          // ...et les lignes. qx[0] : la case collée au A.
```

13. UNE VARIABLE — LONGUEUR

```
uint8_t longueur = 0;    // NOUVEAU : combien de cases la queue utilise (0 : le A seul)
```

14. UNE FONCTION — VIDERECRAN()

```
void viderEcran() {  
    for (uint8_t l = 0; l < 18; l++) {  
        effacer(0, l, 20);  
    }  
}  
  
// ---- Une partie qui commence, TOUT au départ (le 0.91.2) ----  
// Depuis le titre, ET depuis la fin (START : rejouer).
```

15. UNE FONCTION — NOUVELLEPARTIE()

```
void nouvellePartie() {
    x = 5;                      // le A revient à sa place
    y = 8;
    px = 15;                   // le P revient en (15, 8), d'où qu'il soit
    py = 8;
    ax = 5;                    // et sa place d'avant, la même
    ay = 8;
    longueur = 0;              // NOUVEAU : pas de queue, le A est tout seul
    score = 0;                  // pas encore de pièce
    temps = 30;                 // 30 secondes de nouveau
    ecran = 1;                  // on est sur le JEU
    viderEcran();
    poser(px, py, ALPHABET[15]);

    // ---- Le cadre de murs, des X tout autour du terrain (le 0.91.3) ----
    //
    // colonne : 0 1 2 ...      19
    // ligne 0 : X X X X X X ... X X X   le haut : ligne 0, colonnes 0 à 19
    // ligne 1 : X                X
    // ...      X   A           P   X   les côtés : colonnes 0 et 19
    // ligne 15: X                X
    // ligne 16: X X X X X X ... X X X   le bas : ligne 16
    // ligne 17: SCORE 000  TEMPS 030   la ligne 17 reste aux nombres
    //
    // ALPHABET[23] : la 24e lettre (on compte depuis 0), le X.
    for (uint8_t c = 0; c < 20; c++) {
        poser(c, 0, ALPHABET[23]);      // le haut
        poser(c, 16, ALPHABET[23]);     // le bas
    }
    for (uint8_t l = 1; l < 16; l++) {
        poser(0, l, ALPHABET[23]);      // le côté gauche
        poser(19, l, ALPHABET[23]);     // le côté droit
    }

    texte(0, 17, "SCORE");
    texte(11, 17, "TEMPS");
}
```

16. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    couleurTexte(31, 16, 0);
    texteGrand(4, 5, "JEU", 4);
    texte(2, 12, "APPUIE SUR START");

    while (true) {
        image();

        if (ecran == 0 && bouton(START)) {
            nouvellePartie();          // la première partie
        }

        if (ecran == 1) {
            ax = x;                      // on retient où est le A... (le 0.91.3)
            ay = y;
            deplace_croix(x, y, ALPHABET[0], 250); // ...il fait peut-être un pas...

            // ...et s'il est arrivé SUR le cadre (colonne 0 ou 19, ligne 0 ou 16) :
            // 1. on remet le X, que le A venait de recouvrir ;
            // 2. le A revient à sa place d'avant, et s'y pose.
            // || veut dire « OU » : une seule des quatre suffit.
            if (x == 0 || x == 19 || y == 0 || y == 16) {
                poser(x, y, ALPHABET[23]); // le mur revient
                x = ax;                     // le A recule
                y = ay;
                poser(x, y, ALPHABET[0]);
            }

            // ---- La queue suit le A (le 0.91.5) ----
            // Le A a-t-il bougé ? Sa place n'est plus celle d'avant (ax, ay).
            // NOUVEAU : longueur > 0 d'abord. Sans queue, rien à déplacer, et
            // longueur - 1 vaudrait 255 (un uint8_t ne descend pas sous 0).
            // && veut dire « ET » : il faut une queue ET que le A ait bougé.
            if (longueur > 0 && (x != ax || y != ay)) {
                // 1. le BOUT de la queue disparaît : la dernière case, longueur - 1
                effacer(qx[longueur - 1], qy[longueur - 1], 1);

                // ---- NOUVEAU : 2. chaque case prend la place de celle de devant ----
                // En partant du BOUT, et en reculant (i-- : i = i - 1).
                // Avec longueur = 3 : i = 2 : qx[2] = qx[1]
                //                      i = 1 : qx[1] = qx[0]
                //                      i = 0 : 0 > 0 est faux, la boucle s'arrête.
                for (uint8_t i = longueur - 1; i > 0; i--) {
                    qx[i] = qx[i - 1];
                    qy[i] = qy[i - 1];
                }

                // 3. la première case prend la case que le A vient de quitter
                qx[0] = ax;
                qy[0] = ay;

                // 4. toute la queue redessinée, de la case 0 à la case longueur - 1
                // (après une pièce, la nouvelle case venait d'être effacée avec le bout)
                for (uint8_t i = 0; i < longueur; i++) {
                    poser(qx[i], qy[i], ALPHABET[14]);
                }
                poser(x, y, ALPHABET[0]); // 5. le A par-dessus (le 0.91.5)
            }

            if (x == px && y == py) { // le A sur le P : ramassée (le 0.81)
                score = score + 1;

                // ---- La pièce réapparaît AU HASARD, mais DANS le cadre (le 0.91.4) ----
                //
                // px = 1 + hasard() % 18;
```

```

//      |   |   |
//      |   |   +-- % 18 : le reste de la division par 18, de 0 à 17.
//      |   |   Exemple : hasard() rend 137 ; 137 = 7 × 18 + 11 ;
//      |   |   le reste est 11.
//      |   +----- un nombre imprévisible, de 0 à 255 (le 0.81.2)
//      +----- + 1 : on décale de 0...17 à 1...18 → 11 + 1 = 12.
//
// Pourquoi pas % 20 comme au 0.81.2 ? Les colonnes 0 et 19 sont des murs X :
// la pièce doit rester de 1 à 18. Les lignes 0 et 16 aussi sont des murs :
// la pièce doit rester de 1 à 15, d'où 1 + hasard() % 15.
px = 1 + hasard() % 18;           // une colonne : 1 à 18
py = 1 + hasard() % 15;           // une ligne : 1 à 15
poser(px, py, ALPHABET[15]);      // le nouveau P apparaît là

// ---- NOUVEAU : la queue grandit d'une case ----
// La nouvelle case naît SUR le bout de la queue. Au prochain pas, toutes
// les autres avancent, elle non : elle reste derrière.
// La toute première n'a pas de bout : elle naît SOUS le A.
// < 50 : le tableau n'a que 50 cases, qx[0] à qx[49].
if (longueur < 50) {
    if (longueur == 0) {
        qx[0] = x;                 // la première case : sous le A
        qy[0] = y;
    } else {
        qx[longueur] = qx[longueur - 1]; // la case d'après le bout...
        qy[longueur] = qy[longueur - 1]; // ...à la même place que le bout
    }
    longueur = longueur + 1;        // une case de plus
}
}
nombre(6, 17, score);
nombre(17, 17, temps);
}

if (ecran == 1 && chaque(1000)) {
    temps = temps - 1;
    if (temps == 0) {
        ecran = 2;
        viderEcran();
        texte(8, 6, "FIN");
        texte(5, 9, "SCORE");        // le score de la partie (le 0.91.2)
        nombre(11, 9, score);
        texte(2, 13, "START : REJOUER");
    }
}

// ---- Sur la fin, START relance une partie (le 0.91.2).
if (ecran == 2 && bouton(START)) {
    nouvellePartie();                // la même fonction : tout au départ
}
}
}
}

```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



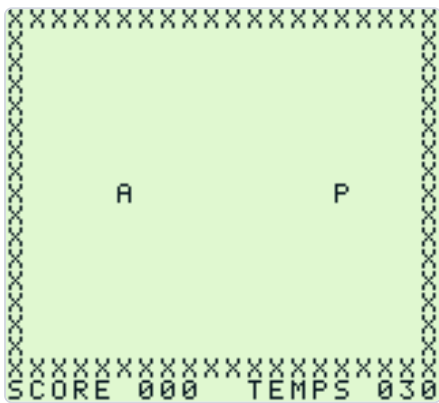
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



START — l'écran a changé

5. Ce qu'on doit voir

START : le A tout seul ; à chaque P ramassé, un O de plus derrière lui.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ : le A tout seul, aucun O
- ✓ le P est ramassé (score = 1)
- ✓ la queue a grandi : une case par pièce (longueur = 1)
- ✓ autant de O à l'écran que de cases (1 O)

Ensuite : **0.91.7. Le snake — le A avance tout seul** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++